

INTRODUCCIÓN A LOS SISTEMAS DISTRIBUIDOS
(75.43) CURSO HAMELIN

TP N°2
Software-Defined Networks

× ×

22 de junio de 2025

Vintoñuke Sebastian Mauricio
106063
Leandro Rodrigo Pesa
110076

Origoni Marcelo Agustin
109903
Santiago Bautista Trezeguet
110343
Agustin Pelliciari
108172

1. Preguntas a Responder

- ¿Cuál es la diferencia entre un Switch y un router? ¿Qué tienen en común?

Un switch opera en la capa de enlace y toma decisiones en función de la dirección MAC. Un router opera en la capa de red y toma decisiones en función de la dirección IP. Ambos dispositivos conmutan datos.

- ¿Cuál es la diferencia entre un Switch convencional y un Switch OpenFlow?

Un switch convencional toma decisiones según un protocolo estático definido por el fabricante. En cambio, un switch OpenFlow permite delegar y definir su lógica de decisión mediante software externo.

- ¿Se pueden reemplazar todos los routers de la Internet por Switches OpenFlow? Piense en el escenario interASes para elaborar su respuesta

En los escenarios interASes los routers utilizan el protocolo BGP para poder coordinar las rutas de forma descentralizada y autónoma. Sin embargo, los Switches OpenFlow requieren de un control centralizado. Reemplazar routers por Switches OpenFlow tendría problemas de escalabilidad e iría en contra de la necesidad de los sistemas autónomos de configurar su enrutamiento de manera autónoma.

2. Topología

Se desarrolló la topología parametrizable planteada, consistente en una cantidad variable de switches conectados en cadena, con un host en cada extremo. Los switches aprenden automáticamente la topología como se especifica en el enunciado.

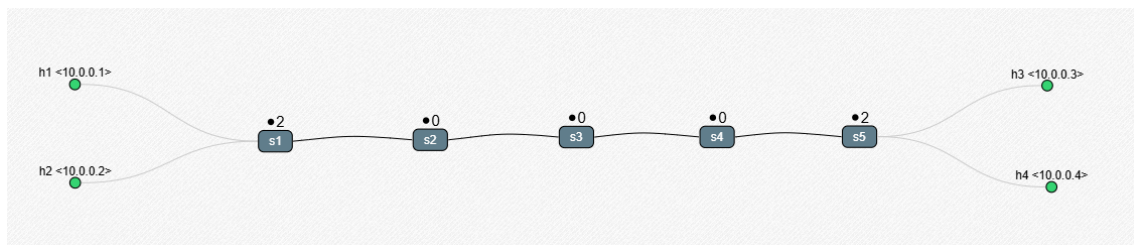


Figura 1: Topología con 5 Switches utilizada en las pruebas.

2.1. Ejecución

El siguiente comando permite ejecutar la topología con $n + 2$ switches:

```
$ sudo mn --custom ./dynamic_topology.py --topo dynamicTopology, 3 --mac --arp  
--switch ovsk --controller remote
```

3. Firewall

Se modificó el controlador para que funcione como un firewall, conforme a las reglas especificadas en el enunciado.

3.1. Reglas Implementadas

Las reglas implementadas fueron las siguientes:

1. Se descartan todos los mensajes cuyo puerto destino sea 80.
2. Se descartan todos los mensajes que provengan del host 1, tengan como puerto destino el 5001, y estén utilizando el protocolo UDP.
3. Dos hosts no pueden comunicarse de ninguna forma.

3.2. Archivo de Reglas

El archivo de reglas fue creado en formato JSON. Un ejemplo del formato utilizado es el siguiente:

```
[
  {
    "dst_port": "80"
  },
  {
    "mac_src": "00:00:00:00:00:01",
    "dst_port": 5001,
    "protocol": "udp"
  },
  {
    "mac_src": "00:00:00:00:00:01",
    "mac_dst": "00:00:00:00:00:04",
    "bidireccional": true
  }
]
```

3.3. Supuestos

1. Si se bloquea por puerto y no se especifica el protocolo de transporte, se asume tanto UDP como TCP.
2. Si se bloquea por protocolo de transporte y no se especifica el protocolo de red, se asume solo IPv4.
3. Si se bloquea por IPv6, no se pueden especificar bloqueos por protocolo de transporte.

3.4. Ejecución

El siguiente comando permite ejecutar el firewall:

```
$ python ./pox.py --verbose samples.spanning_tree redes.tp2.firewall
```

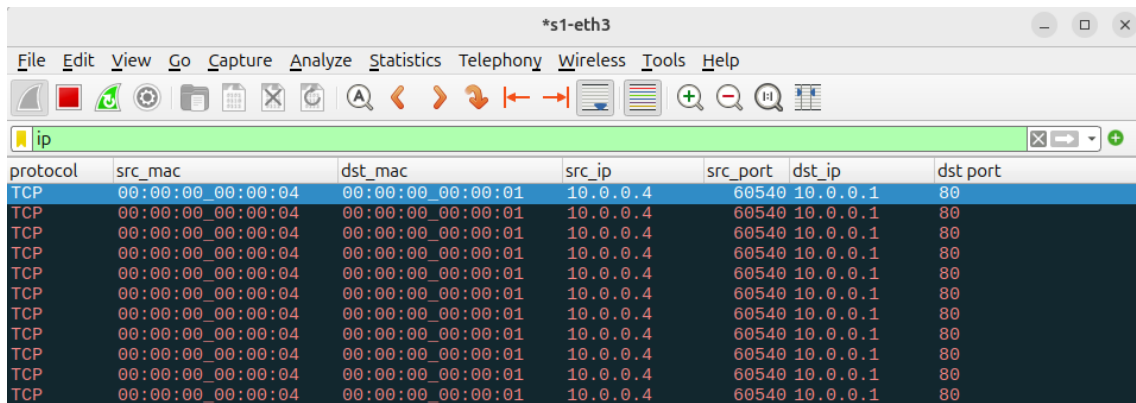
4. Wireshark

Se realizaron pruebas simulando tráfico con iPerf sobre los casos especificados en el enunciado

4.1. Se descartan todos los mensajes cuyo puerto destino sea 80

```
[redes.tp2.firewall] Creating new block rule:  
[redes.tp2.firewall] dst port: 80  
[redes.tp2.firewall] Loaded Rules got rules count 1
```

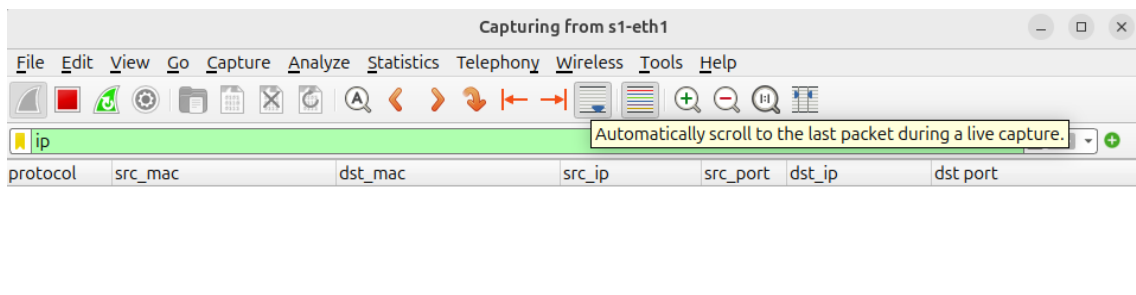
Figura 2: Se configura un bloqueo por puerto en el switch entre las interfaces `eth1` y `eth3`.



The image shows a Wireshark capture window titled '*s1-eth3'. The filter bar contains 'ip'. The packet list shows 14 TCP packets, all with destination port 80. The packet details pane shows the selected packet's structure: Ethernet II, Internet Protocol Version 4, and Transmission Control Protocol.

protocol	src_mac	dst_mac	src_ip	src_port	dst_ip	dst port
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	60540	10.0.0.1	80

Figura 3: Los paquetes pasan por `eth3`.



The image shows a Wireshark capture window titled 'Capturing from s1-eth1'. The filter bar contains 'ip'. The packet list is empty, indicating that no packets are being captured on this interface.

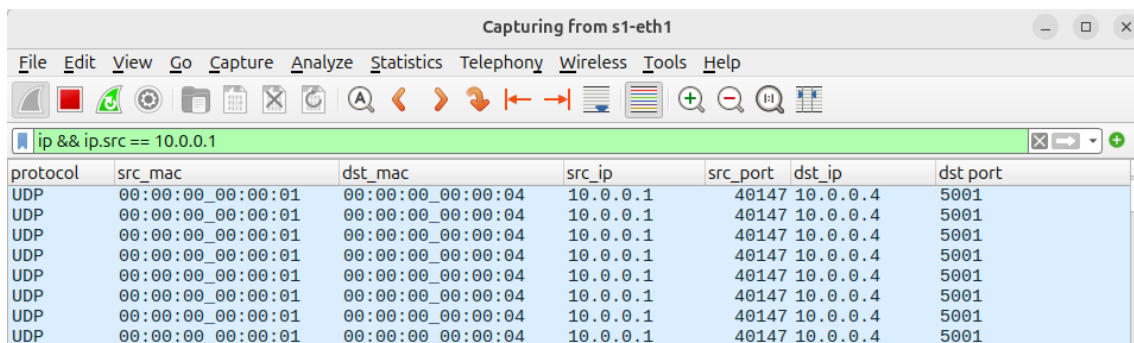
protocol	src_mac	dst_mac	src_ip	src_port	dst_ip	dst port
----------	---------	---------	--------	----------	--------	----------

Figura 4: Los paquetes no pasan por `eth1`.

- 4.2. Se descartan todos los mensajes que provengan del host 1, tengan como puerto destino el 5001, y estén utilizando el protocolo UDP

```
[redes.tp2.firewall] Creating new block rule:  
[redes.tp2.firewall] transport protocol: udp  
[redes.tp2.firewall] src mac: 00:00:00:00:00:01  
[redes.tp2.firewall] dst port: 5001  
[redes.tp2.firewall] Loaded Rules got rules count 1  
[redes.tp2.firewall] Attaching Redes Firewall to switch: id:00-00-00-00-00-01
```

Figura 5: Se configura un bloqueo por dirección MAC, puerto y protocolo sobre el switch entre las interfaces `eth1` y `eth3`.



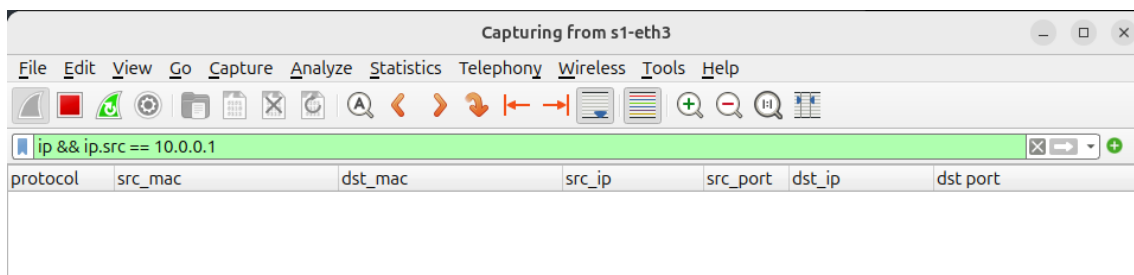
Capturing from s1-eth1

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip && ip.src == 10.0.0.1

protocol	src_mac	dst_mac	src_ip	src_port	dst_ip	dst port
UDP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	40147	10.0.0.4	5001
UDP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	40147	10.0.0.4	5001
UDP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	40147	10.0.0.4	5001
UDP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	40147	10.0.0.4	5001
UDP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	40147	10.0.0.4	5001
UDP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	40147	10.0.0.4	5001
UDP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	40147	10.0.0.4	5001
UDP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	40147	10.0.0.4	5001

Figura 6: Los paquetes pasan por `eth1`.



Capturing from s1-eth3

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

ip && ip.src == 10.0.0.1

protocol	src_mac	dst_mac	src_ip	src_port	dst_ip	dst port
----------	---------	---------	--------	----------	--------	----------

Figura 7: Los paquetes no pasan por `eth3`.

4.3. Dos hosts no pueden comunicarse de ninguna forma

```
[redes.tp2.firewall] Creating new block rule:
[redes.tp2.firewall] src mac: 00:00:00:00:00:01
[redes.tp2.firewall] dst mac: 00:00:00:00:00:04
[redes.tp2.firewall] Adding inverted rule:
[redes.tp2.firewall] dst mac: 00:00:00:00:00:01
[redes.tp2.firewall] src mac: 00:00:00:00:00:04
[redes.tp2.firewall] Loaded Rules got rules count 2
[redes.tp2.firewall] Attaching Redes Firewall to switch: id:00-00-00-00-00-01
```

Figura 8: Se configura un bloqueo bidireccional por dirección MAC sobre el switch entre las interfaces eth1 y eth3.

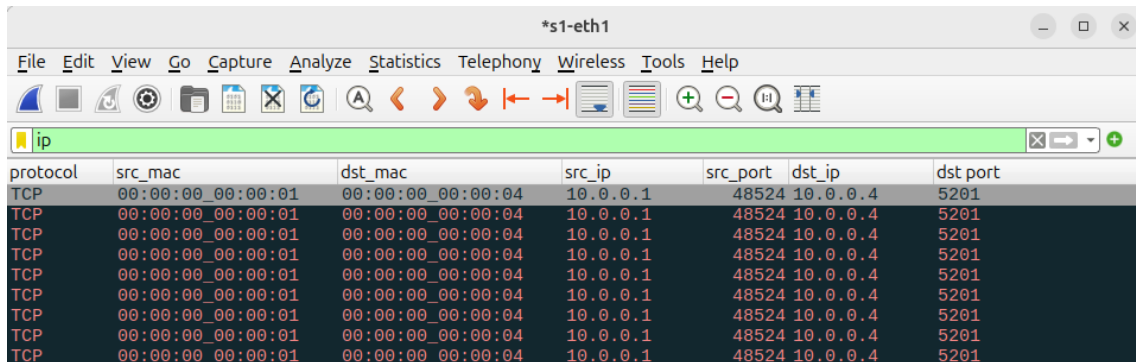


Figure 9 shows a Wireshark capture on interface *s1-eth1. The capture filter is 'ip'. The table below represents the data shown in the packet list pane.

protocol	src_mac	dst_mac	src_ip	src_port	dst_ip	dst port
TCP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	48524	10.0.0.4	5201
TCP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	48524	10.0.0.4	5201
TCP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	48524	10.0.0.4	5201
TCP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	48524	10.0.0.4	5201
TCP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	48524	10.0.0.4	5201
TCP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	48524	10.0.0.4	5201
TCP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	48524	10.0.0.4	5201
TCP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	48524	10.0.0.4	5201
TCP	00:00:00_00:00:01	00:00:00_00:00:04	10.0.0.1	48524	10.0.0.4	5201

Figura 9: Los paquetes que van de h1 a h4 pasan por eth1, pero no por eth3.

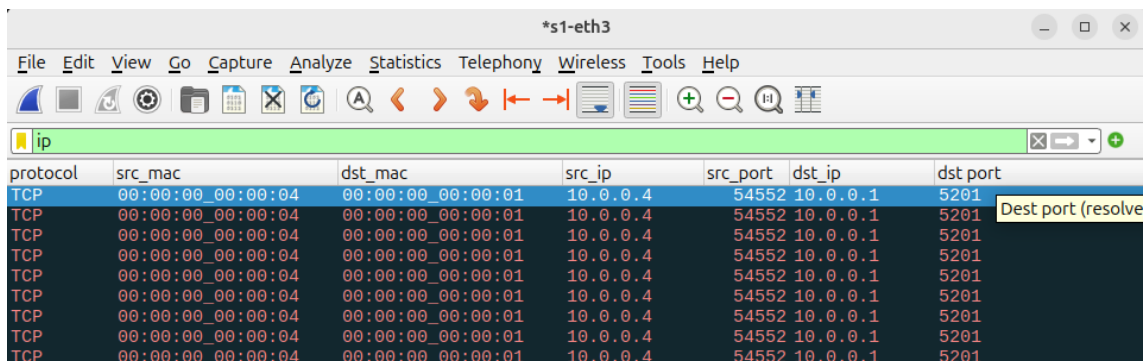


Figure 10 shows a Wireshark capture on interface *s1-eth3. The capture filter is 'ip'. The table below represents the data shown in the packet list pane.

protocol	src_mac	dst_mac	src_ip	src_port	dst_ip	dst port
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	54552	10.0.0.1	5201
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	54552	10.0.0.1	5201
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	54552	10.0.0.1	5201
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	54552	10.0.0.1	5201
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	54552	10.0.0.1	5201
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	54552	10.0.0.1	5201
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	54552	10.0.0.1	5201
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	54552	10.0.0.1	5201
TCP	00:00:00_00:00:04	00:00:00_00:00:01	10.0.0.4	54552	10.0.0.1	5201

Figura 10: Los paquetes que van de h4 a h1 pasan por eth3, pero no por eth1.